

Contents

Step 07 — Opponent Modeling: Exploration & Implementation Report	1
PART I — EXPLORATION PHASE (Phase 2)	2
1. What this step is about	2
2. Experiment 1 — Behavioral fingerprints	3
3. Experiment 2 — The exploitation opportunity (why bother)	3
4. Experiment 3 — The Bayesian type detector (the core loop)	4
5. Experiment 4 — Recovering the blend (mixture modeling with EM)	8
6. Why the exploration phase matters for the thesis	10
7. Methodology note — prediction vs. observed reality	11
PART II — IMPLEMENTATION PHASE (Phase 4)	11
8. What was built	11
9. Correctness — <code>validate.py</code> (8 / 8 PASS)	12
10. Detection — who is the opponent?	13
11. Exploitation — how much does modeling win?	13
12. Non-stationarity — the two-sided finding	15
13. Were the samples large enough? — statistical adequacy	17
14. Bugs found and fixed (first-execution engineering record)	18
15. Concerns and threats to validity	19
16. New research directions this opened	20
REPRODUCTION	20
17. Reproduction	20

Step 07 — Opponent Modeling: Exploration & Implementation Report

Environment: July 2026 **Games:** Kuhn Poker (3-card, 2-player) and Leduc Hold'em (6-card, 2-round) — the smallest exact-solvable imperfect-information testbeds, imported from Steps 02/03 **Phases covered:** Phase 2 (Exploration) and Phase 4 (Implementation) **Foundation reused:** the validated Step 02 Kuhn engine + CFR solver and the Step 03 Leduc engine (imported, never copied) **PhD connection: Contribution #1 (Behavioral Adaptation Framework)** — the opponent model is the *sensor* (turning observed actions into an estimate of the opponent's strategy); the adaptive exploiter is a first *actuator*, which Step 08 formalizes into safe (KL-regularized) exploitation. **Status:** Exploration — all experiments run and verified. Implementation — `validate.py` **8/8 PASS**; the full **scale** tournament ran end-to-end on both games (Kuhn ≈ 2 min, Leduc **826.8 s ≈ 14 min**; 5 seeds, 20k-hand matches, 200k/40k-iter Nash). Every number below is a **measured result** from a real run on this machine, not a prediction.

Document structure. This report covers Step 07 end-to-end in two parts. **Part I (Exploration, §1–§7)** is the seeded, fast intuition-building phase on Kuhn Poker — *what* an opponent type looks like, *why* it is worth modeling, and *how* the belief-update loop be-

haves (including where it breaks). **Part II (Implementation, §8–§16)** is the full Bayesian opponent-modeling + adaptive-exploitation build, run and validated on both games against exact analytical yardsticks. §17 collects reproduction commands for both.

PART I — EXPLORATION PHASE (Phase 2)

1. What this step is about

A **Nash-equilibrium** strategy is built never to lose in the long run, *no matter who it plays*. That safety has a price: it plays identically against a world champion and against someone who folds every time you bet. **Opponent modeling** is the act of *watching how a specific opponent actually plays* and updating a belief about their strategy, so we can **deviate from Nash to exploit their mistakes** — bluff more against someone who folds too much, value-bet thinner against someone who calls too much.

The technique explored here is the **explicit type-based (Bayesian) model**: instead of learning a strategy from a blank slate, we keep a belief — a probability distribution — over a small set of predefined opponent **types**, and update it after every observed action via Bayes’ rule:

$$\text{posterior}(\text{type}) \propto \text{prior}(\text{type}) \times P(\text{observed action} \mid \text{that type})$$

An action a type calls impossible drives that type toward zero; an action it predicts well boosts it. Over many hands the belief concentrates on the best-fitting type, and we best-respond to it.

This part covers the **exploration phase**: three small Kuhn-Poker experiments that make the idea concrete. Every number below is a **measured result** from a real run (seeded, reproducible), not a prediction.

1.1 The opponent “type zoo”

Four ground-truth types serve both as the opponents we play against and as the candidate hypotheses the detector reasons over:

Type	Behavior
AlwaysCall	Calling station — never bets, never folds (checks an open pot, calls any bet).
TightPassive	Rock — only commits chips with the nuts (K); folds everything else. The most exploitable type.
LooseAggressive	Maniac — always bets / calls, regardless of hand.
Nash	Balanced equilibrium play (from the Step 02 CFR solver); mixes its actions; unexploitable.

2. Experiment 1 — Behavioral fingerprints

Question: does each type leave a distinct, recoverable signature in its actions?

Method: play each type against a 50/50 “prober” (to force every information set to be visited) and record the empirical $P(\text{bet})$ at each info set next to the type’s true $P(\text{bet})$.

Result. The four fingerprints are visibly distinct and the empirical frequencies track the true probabilities at well-visited info sets:

- **LooseAggressive** ≈ 1.0 everywhere; **TightPassive** ≈ 0 except with K; **AlwaysCall** ≈ 0 when checking into an open pot but ≈ 1.0 when facing a bet; **Nash** fractional and mixed.

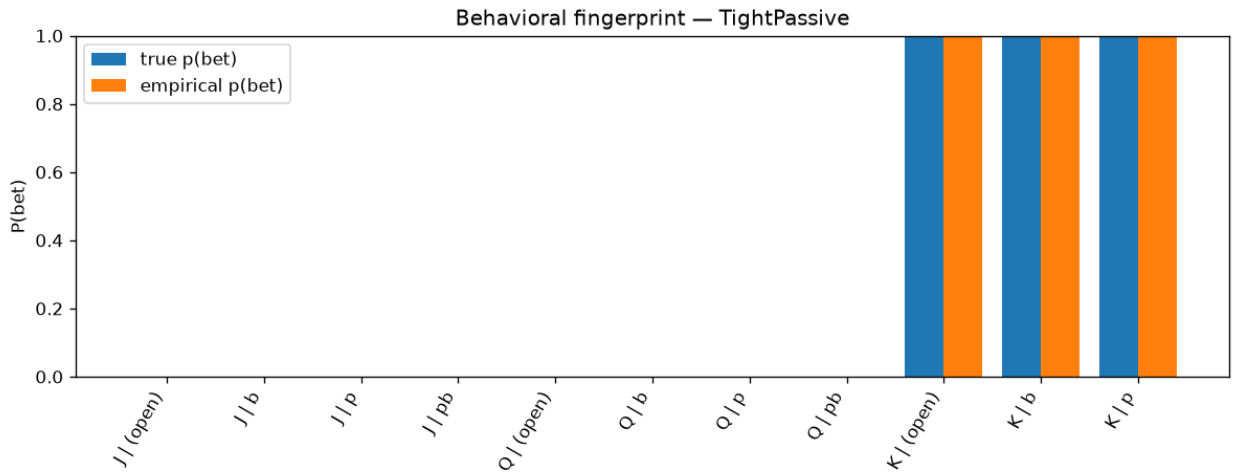


Figure 1: Behavioral fingerprint — TightPassive (bets only with the King).

Takeaway. Recovering a type’s fingerprint from observed actions is the whole modeling task, and it is feasible — but info sets with few visits have noisy empirical frequencies, which is exactly what makes modeling hard on little data.

3. Experiment 2 — The exploitation opportunity (why bother)

Question: how much value does modeling actually unlock over blind Nash play?

Method: for each type and seat, compute *exactly* (i) the Nash hero’s expected value, (ii) the best-response EV (the exploitation ceiling, assuming the type is known perfectly), and (iii) the gap between them. The gap is the money Nash leaves on the table.

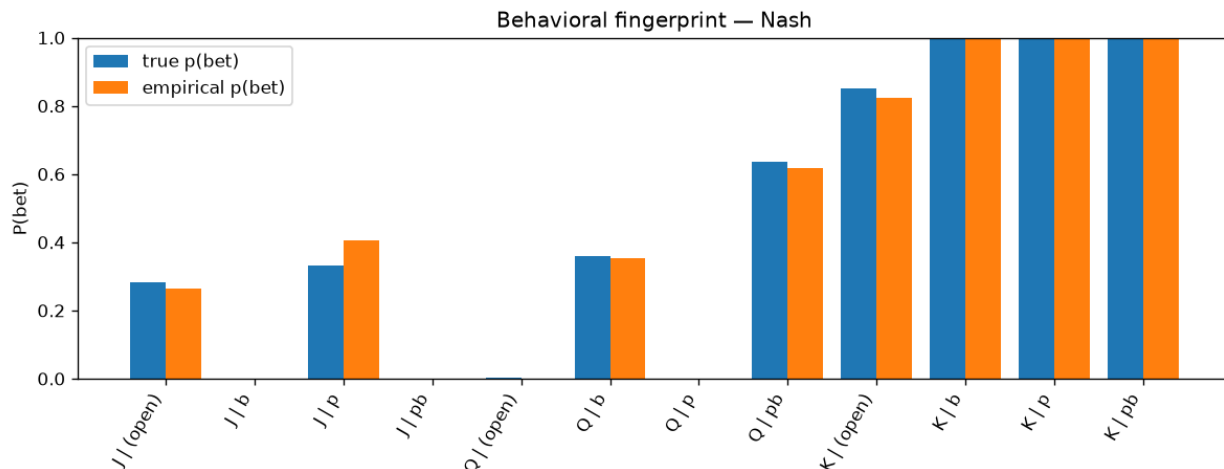


Figure 2: Behavioral fingerprint — Nash (mixed frequencies; empirical tracks true).

Opponent	Seat	Nash EV	Best-response EV	Gap
AlwaysCall	0	+0.119	+0.333	0.215
AlwaysCall	1	+0.220	+0.333	0.113
TightPassive	0	-0.060	+0.167	0.227
TightPassive	1	+0.055	+0.333	0.278
LooseAggressive	0	+0.121	+0.333	0.213
LooseAggressive	1	+0.116	+0.333	0.217
Nash	0	-0.055	-0.052	0.004
Nash	1	+0.055	+0.060	0.005

Result. Against the three exploitable types the gap is large (0.11–0.28 per hand); against **Nash the gap is ≈ 0** — as it must be, since Nash is unexploitable and best response cannot beat the game value. Concretely, the computed best response to **TightPassive bluffs**: it bets the weakest hand (Jack) into an open pot, because that type folds everything but the King.

Takeaway. The value of opponent modeling is real and measurable, and exploiting a rock means *bluffing more*. (The $-1/18 \approx -0.056$ seat-0 disadvantage of Kuhn’s first player explains the small negative Nash EVs — that is the game, not an error.)

4. Experiment 3 — The Bayesian type detector (the core loop)

Question: starting from an even prior, can the belief-update loop identify the opponent from behavior alone — and what happens when the opponent matches *no* type?

Method: maintain a posterior over the four types; after each observed opponent action multiply in its likelihood under each type and renormalize (in log-space, with a small $\epsilon = 0.02$ likelihood smoothing so no single “impossible” action permanently kills a candidate). Run several hidden opponents and watch the posterior evolve hand by hand.

4.1 In-set opponents — fast, correct, stable

When the hidden opponent genuinely **is** one of the four candidates, the posterior concentrates correctly and locks in:

- **Hidden = TightPassive:** posterior piles onto TightPassive and locks at ≈ 1.0 by hand ~ 18 . (Nash is the slow-to-die rival, because a rock and equilibrium play overlap on many hands.)
- **Hidden = AlwaysCall:** locks by hand ~ 4 (a very distinctive type).

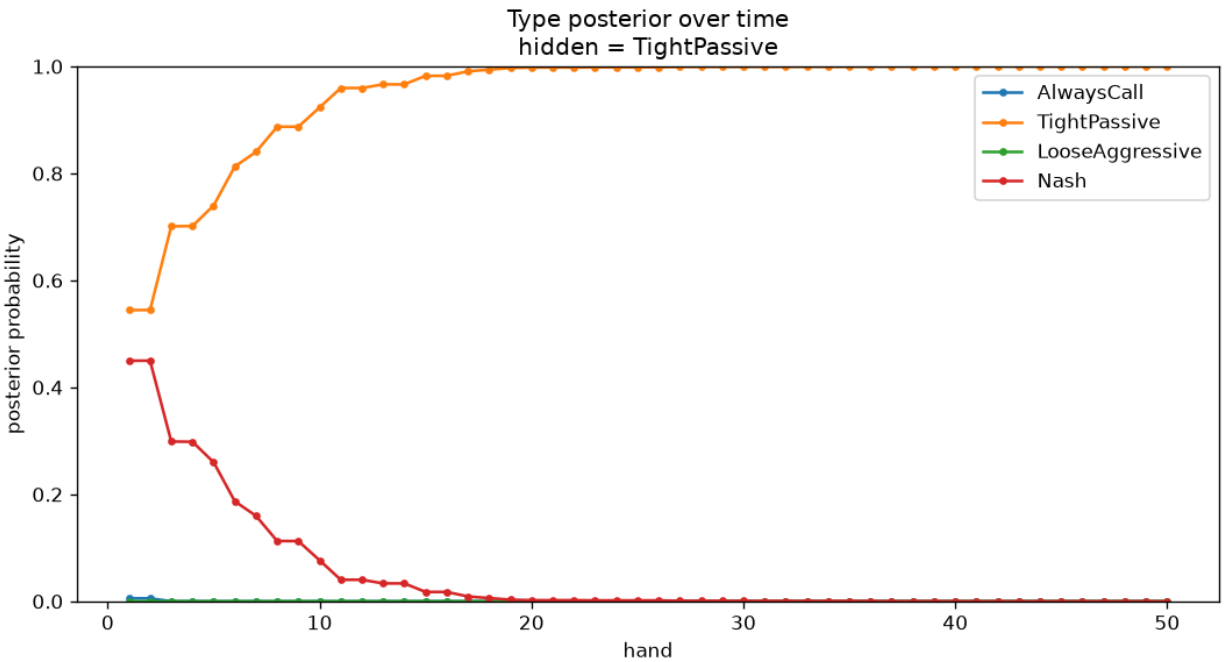


Figure 3: Hidden = TightPassive: clean convergence to the correct type.

4.2 Out-of-set opponent — “no honest home”

The instructive case is a hidden opponent that is a **50/50 per-action blend** of TightPassive and LooseAggressive — deliberately *none* of the four types. The naive expectation was that the posterior would “split between the two nearest types.” **It does not.** The detector commits hard to a *single* type at a time and jumps:

AlwaysCall (hands 1–2) → LooseAggressive (hands ~ 5 –40) → **Nash (hand $\sim 43+$, ≈ 1.0).**

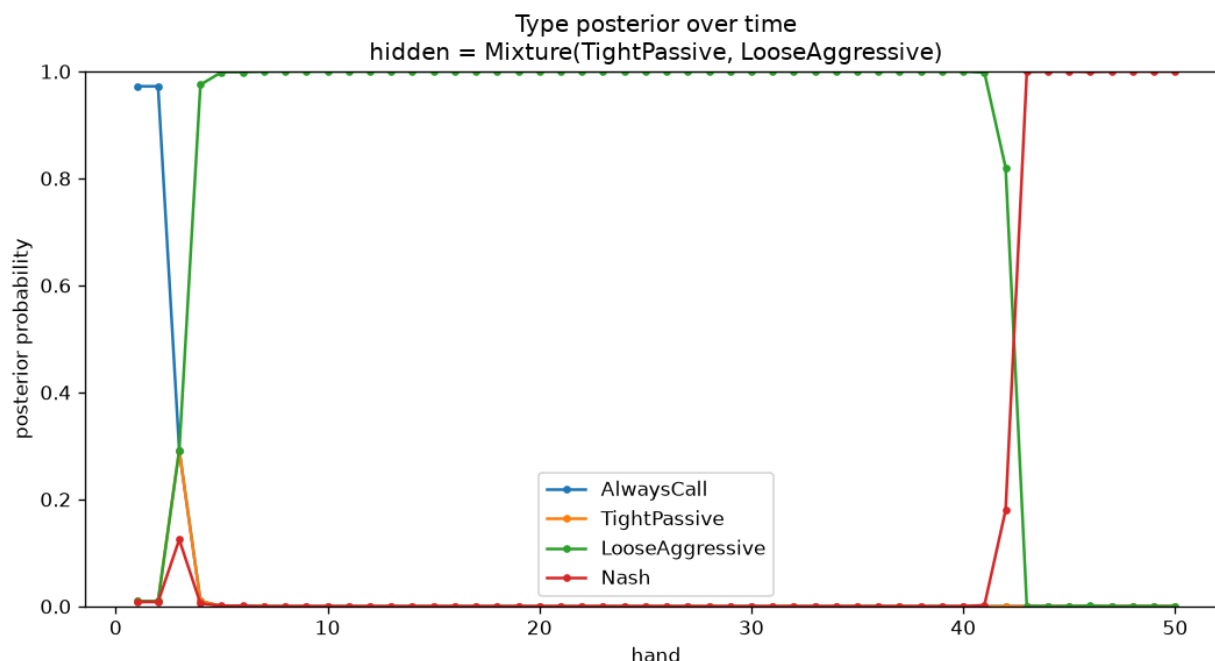


Figure 4: Hidden = Mixture(TightPassive, LooseAggressive): the belief thrashes, then lands on Nash.

This is *correct* Bayesian behavior, not a bug — and the reason is worth stating precisely, because it is the central lesson of the phase:

- The blend plays a true $(0.5, 0.5)$ on middling hands (J/Q) and always bets the King. Every time it bets a J/Q, the rock (which never does) takes a likelihood penalty; every time it checks a J/Q, the maniac (which always bets) takes one.
- The two deterministic extremes are therefore each contradicted about half the time. **Nash is the only candidate that assigns real probability to *both* betting and checking a middling hand**, so it is the only story never fatally contradicted — and a product-of-likelihoods posterior concentrates on the single best explanation, not a blend.

Takeaway. A four-stereotype model has *no honest way to say “none of the above.”* Faced with an opponent it cannot represent, it commits — confidently — to the nearest *mixed* type rather than reporting genuine uncertainty. Two things are worth stressing. First, the posterior is a **relative, normalized** quantity — the four bars are forced to sum to 1, so $P(\text{Nash})=1.0$ means “best fit *among these four*,” **not** “good fit in absolute terms.” Measuring the winner’s absolute fit (geometric-mean probability it assigns to each observed action) exposes the difference: the winning Nash model scores **0.74 per action against a true Nash opponent but only 0.31 against the mixture** — equally confident on paper, but a far worse fit underneath. Second, the honest fix is to stop asking “which one type?” and start asking “what *mixture* of types?” — which is Experiment 4 (§5), and the motivation for the **continuous** and **consistent** models built in the implementation phase.

4.3 How robust is that convergence? — a 500-hand stress test

Because §4.2 lands on Nash, a natural question is: *over a long match, can the belief still “fall” away from Nash?* We ran the mixture scenario for **500 hands across 300 random seeds** and separated two phenomena that a naive metric conflates — *slow convergence* versus *falling after convergence*.

Measurement (300 seeds × 500 hands)	Result
Final winner at hand 500 is Nash	300 / 300 (100%)
After Nash <i>permanently</i> takes the lead, it later dips below 0.5	0 / 300 (never)
A <i>wrong</i> type still led past hand 100	40 / 300 (~13%)
A <i>wrong</i> type still led past hand 200	14 / 300 (~5%)
Hand at which Nash locks in for good	median 23 · 90th-pct 125 · worst 461

Result. Nash never *falls*: once it takes the lead for good it stays pinned at ≈ 1.0 . The real risk is the opposite — **slow convergence with a confident wrong answer in the meantime**. In a sizeable minority of runs the maniac (LooseAggressive) owns the belief at ≈ 1.0 for well over a hundred hands before Nash overtakes, purely because an early run of bet-heavy hands looked exactly like “always bets.” The chart below (the most volatile seed found) shows LooseAggressive holding ≈ 1.0 from hand ~ 35 to ~ 220 , then a sharp cliff to Nash for the remainder:

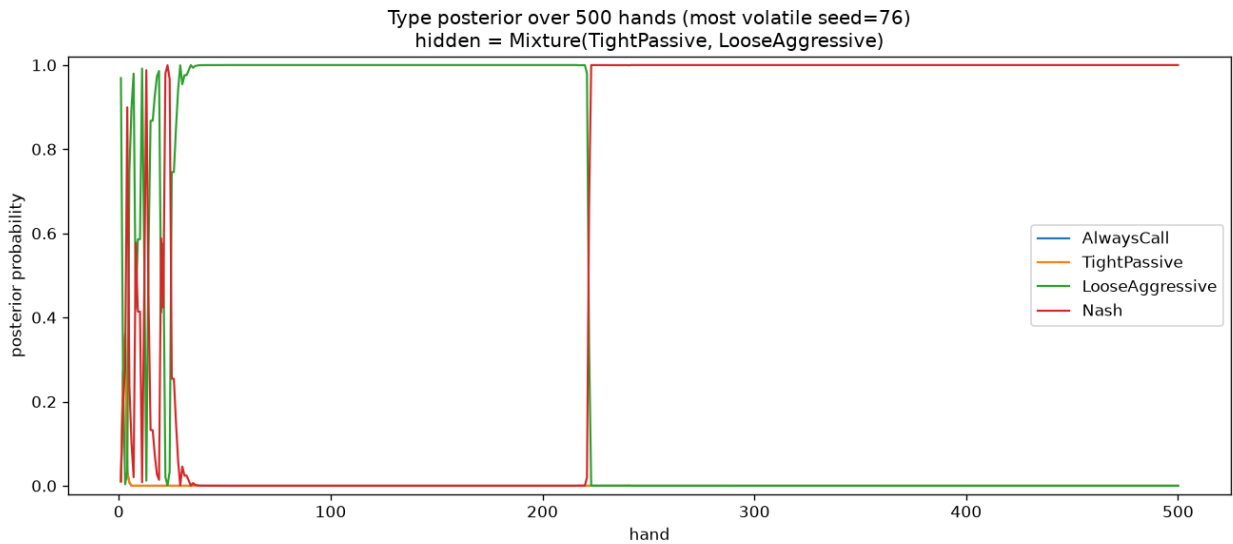


Figure 5: Hidden = Mixture, 500 hands (most volatile seed): a wrong type can dominate for 200 hands before Nash overtakes; once locked, Nash never falls.

Why it can never end wrong. Among the four candidates, Nash is the one *closest* (minimum KL divergence) to the true blended strategy, so every deterministic type eventually meets a contradiction it cannot survive. Nash is the unique long-run winner — “long run” just occasionally means 200+ hands.

5. Experiment 4 — Recovering the blend (mixture modeling with EM)

Question: §4 showed that asking “*which single type are you?*” collapses a blended opponent onto the nearest mixed type (Nash) and hides a poor absolute fit. Can we instead ask “*what **mixture** of types are you?*” and recover the actual blend?

Method. Fit **mixing weights** π over the same four fixed types by **Expectation–Maximization**: each observed action is *softly credited* to the types in proportion to how well each explains it (E-step), and the weights are the average credit (M-step), iterated to convergence.

- E-step: $\text{responsibility}(\text{type} \mid \text{action}) \propto \pi(\text{type}) \times P(\text{action} \mid \text{type})$
- M-step: $\pi(\text{type}) = \text{mean responsibility over all observations}$

A tempting shortcut — a **hard** per-hand tally (pick the single best-fitting type each hand, then count) — does **not** work: it is confounded by *type overlap* (e.g. AlwaysCall and TightPassive both check a weak hand, so the argmax cannot separate them). The soft, iterated EM version deconfounds them.

Result. We re-fit the weights after every hand (warm-started from the previous estimate) to get a **per-hand trajectory** — the mixture analogue of §4’s posterior-over-time charts. The two active components climb to their true weights while the other two decay to zero, whereas the old single-type posterior instead reports Nash ≈ 1.0 throughout:

Hidden opponent	Method	AlwaysCall	TightPassive	LooseAggressive	Nash
50 / 50 blend	global	0.00	0.00	0.00	1.00 ✗
	posterior (old)				
	EM weights @250 hands	0.00	0.51	0.49	0.00 ✓
	<i>true blend</i>	0.00	<i>0.50</i>	<i>0.50</i>	0.00
70 / 30 blend	global	0.00	0.00	0.00	1.00 ✗
	posterior (old)				
	EM weights @250 hands	0.00	0.66	0.34	0.00 ✓

Hidden opponent	Method	AlwaysCall	TightPassive	LooseAggressive	Nash
	<i>true blend</i>	0.00	0.70	0.30	0.00

(The small residual error at 250 hands is sampling noise; a longer run tightens it — 4,000 hands gives 0.50/0.50 and 0.69/0.31.)

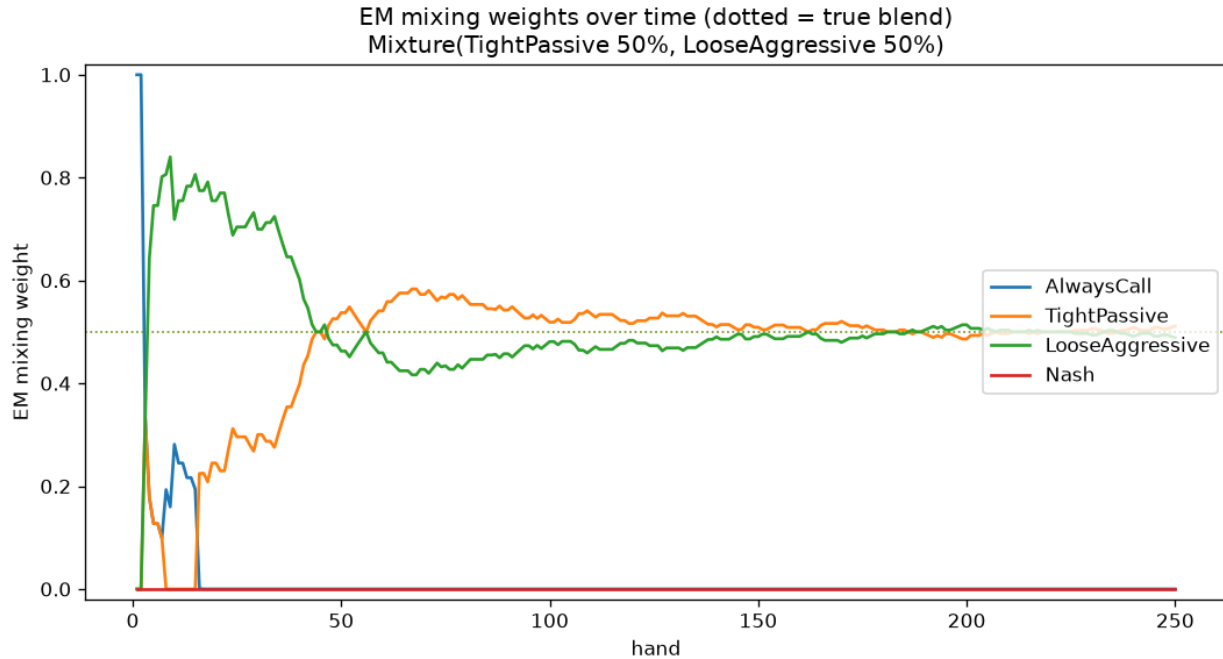


Figure 6: Recovering a 50/50 blend — EM mixing weights per hand. After early thrashing, TightPassive (orange) and LooseAggressive (green) converge together onto the true 0.50 line while AlwaysCall and Nash decay to zero.

Takeaway. EM recovers not only *which* types are present but *in what proportion* — an interpretable, honest description (“ $\approx 70\%$ rock + 30% maniac”) where the single-type model gave a confident wrong answer. This is standard practice: mixture models fit by EM are the classical tool for “the data is a blend of latent sources,” and opponent modeling’s foundational work (Bayes’ Bluff, Southey et al. 2005) already maintains a *distribution over* opponent strategies rather than committing to one. The implementation phase generalizes this further — a **continuous** per-information-set estimator (a Dirichlet count per situation, which represents *any* strategy including a blend) and a **consistent** sequence-form estimator (Ganzfried 2025) with convergence guarantees. Experiment 4 is the smallest, most readable rung on that ladder.

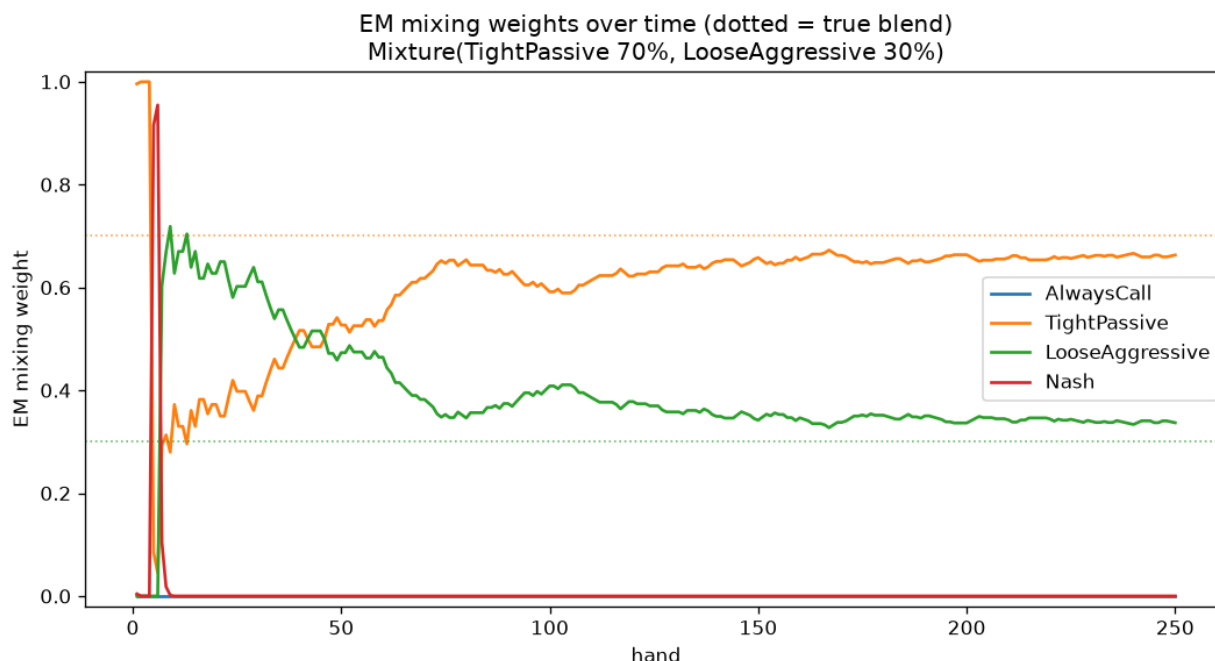


Figure 7: Recovering a 70/30 blend — the two active components settle onto their distinct true weights (dotted), recovering not just *which* two types but their proportions.

6. Why the exploration phase matters for the thesis

The exploration phase already surfaces the tension that Steps 07–08 exist to resolve, and gives it a concrete, measured face:

1. **The value is real (Exp. 2):** blind Nash leaves 0.1–0.3 per hand on the table against exploitable opponents — modeling is worth doing.
2. **But a model can be confident and wrong for a long time (Exp. 3):** in ~13% of 500-hand runs the detector confidently believed the wrong type past hand 100. Best-responding hard to that belief would mean adjusting to beat a phantom — and, per the safety half of the dial, *making ourselves exploitable in the process*.
3. **This is the direct argument for Step 08’s safe exploitation.** One must not best-respond to whatever the model currently believes; the lean toward exploitation has to be regularized (e.g. KL-bounded) and scaled to how *earned* the read is. The exploration phase turns that abstract principle into a demonstrated failure mode.

Part I covers the **sensor** (building the read); Part II builds and validates it in full, and Step 08 builds the **actuator** (exploiting it without becoming exploitable).

7. Methodology note — prediction vs. observed reality

Per the project’s implementation workflow, “expected outcomes” written before a run are *predictions to verify*, not results. Two predictions in this phase did not survive contact with a real run: the Experiment-3 “split between the two nearest types” (§4.2), and an implicit assumption that a confident belief is a settled one (§4.3). In both cases we first ruled out an implementation bug, confirmed the observed behavior was the mathematically correct outcome, and then **kept the original prediction on record alongside an explanation of what really happened and why the confusion arose**. The gap between prediction and reality was the most instructive part of the phase: the “split” the single-type detector *couldn’t* produce (§4.2) is exactly what the mixture model *does* produce once the question is reframed (§5) — the failed prediction directly motivated the follow-up experiment. That is the point of running the experiments at all.

PART II — IMPLEMENTATION PHASE (Phase 4)

Note on process. Per `implementation/WORKFLOW.md`, the implementation code was *written but never run* by the agent that authored it. Part II covers **running, debugging, and validating** it. Every number below is a **measured result** from a real run on this machine. Two correctness bugs and one tractability problem were found and fixed during execution (§14). The OpenSpiel cross-check was skipped (no `pyspiel` on this Windows environment); the exact internal Nash checks substitute.

8. What was built

One uniform interface — *consume observed hands* → *emit a predicted opponent policy* — with **three opponent models** behind it, an **adaptive exploiter** that best-responds to the model (with a Nash safety blend and optional change-point forgetting), and a **validation harness + tournament** that measure everything against exact analytical yardsticks.

Component	What it is
<code>engines.py / policies.py</code>	Uniform Game adapter over the Kuhn & Leduc engines; the policy → {action: prob} currency; replay for partial-observability reasoning.
<code>opponent_types.py / level_k.py</code>	The type “zoo” per game + Nash + Random + Level-k cognitive-hierarchy opponents.

Component	What it is
<code>observation_buffer.py</code> / <code>inference.py</code>	The single home of partial observability — marginalizing the opponent’s hidden card over consistent deals; the eps-smoothed marginal log-likelihood.
<code>type_based_model.py</code>	Dirichlet-multinomial posterior over K types; MAP type + model-averaged policy.
<code>continuous_model.py</code>	Per-information-set Dirichlet estimate (showdowns → hard counts, folds → soft/EM counts).
<code>consistent_model.py</code>	Ganzfried (2025) sequence-form MAP via a SciPy convex solve.
<code>best_response.py</code> / <code>nash.py</code>	Exact info-set-constrained best response and <code>nash_gap</code> (=NashConv); CFR Nash baseline.
<code>adaptive_exploiter.py</code> / <code>changepoint.py</code>	observe→model→BR→act loop with a Nash safety blend and Bayesian online change-point detection.
<code>validate.py</code> / <code>tournament.py</code> / <code>plotting.py</code>	The verification harness (PASS/FAIL vs the raw step’s targets) and the headline experiment + plots.

The three models share one exact best-response, so exploitation is compared apples-to-apples; every learning curve is bounded by an **exact** `nash_ev` (equilibrium can’t be exploited) and `ceiling` (true best-response value).

9. Correctness — `validate.py` (8 / 8 PASS)

Check	Result
Kuhn Nash value (−1/18)	<code>exact_value(Nash,Nash)</code> = −0.0556 ✓
Kuhn Nash unexploitable	<code>NashConv</code> = 0.0020 (< 0.05) ✓
BR beats uniform (both games)	Kuhn +0.500, Leduc +2.087 ✓
Type-based detection	<code>posterior[true]</code> = 1.000, MAP correct ✓
Continuous recovers strategy	mean TV = 0.016 (< 0.15) ✓
Consistent recovers strategy	<code>P(BET King)</code> = 0.987 (> 0.75) ✓
Exploiter beats Nash, under ceiling	realized +0.165, <code>nash_ev</code> −0.047, <code>ceiling</code> +0.167 ✓
Change-point helps non-stationary	post-switch: static −0.181, changepoint +0.211 ✓

Check	Result
-------	--------

The exact Nash checks (value = $-1/18$, NashConv ≈ 0) stand in for the OpenSpiel cross-validation, which is unavailable here.

10. Detection — who is the opponent?

Both games, all types. **Type-based detection is essentially perfect**: the posterior concentrates fully on the true type and the MAP is correct (total-variation distance 0.000) — with **one instructive exception**. On Kuhn the higher Level-k opponents collide: Level-2 and Level-3 converge to the *same* behavioral policy, so the type posterior splits 0.50/0.50 between them and the MAP for the true Level-3 lands on Level-2 (wrong). This is not a bug — two types that play identically are genuinely indistinguishable *by their actions*, which is exactly what a type detector reasons over. The **continuous** and **consistent** models, which estimate the actual policy rather than a label, recover both cleanly (TV ≈ 0.005).

The **continuous** model’s mean TV to the truth is small on Kuhn (0.006–0.030) but larger on Leduc (0.11–0.36) — as predicted: Leduc has far more information sets and heavier partial observability, so a free-form per-info-set estimator needs many more hands to converge; rarely-visited info sets dominate the residual error. The **consistent** model (Kuhn detection) matches or beats the continuous model’s recovery (TV ≈ 0.004 –0.021), as its sequence-form MAP is designed to.

11. Exploitation — how much does modeling win?

Realized mean profit per hand over 20k-hand matches ($\times 5$ seeds) against the exact best-response **ceiling**. The type-based model tracks the ceiling almost exactly on both games; the continuous model matches it on Kuhn and tracks close on Leduc, sitting a little below for the hardest-to-fit types (undersampling, as in §10). *(The consistent model refits a convex program and is impractical in a per-refit online match, so exploitation uses type-based + continuous; the consistent model is evaluated where its strength lies — recovery/detection. See §14.3.)*

Kuhn (ceiling / type-based / continuous):

Opponent	ceiling	type-based	continuous
AlwaysPass	0.975	0.966	0.966
AlwaysBet	0.333	0.335	0.335
LooseAggressive	0.333	0.337	0.330

Opponent	ceiling	type-based	continuous
TightPassive	0.167	0.168	0.168
Nash	-0.055	-0.055	-0.053

Leduc (ceiling / type-based / continuous):

Opponent	ceiling	type-based	continuous
Level1	3.056	3.061	2.672
Maniac	2.177	2.199	2.038
CallingStation	1.464	1.451	1.434
Rock	0.937	0.912	0.848
Nash	-0.083	-0.085	-0.175

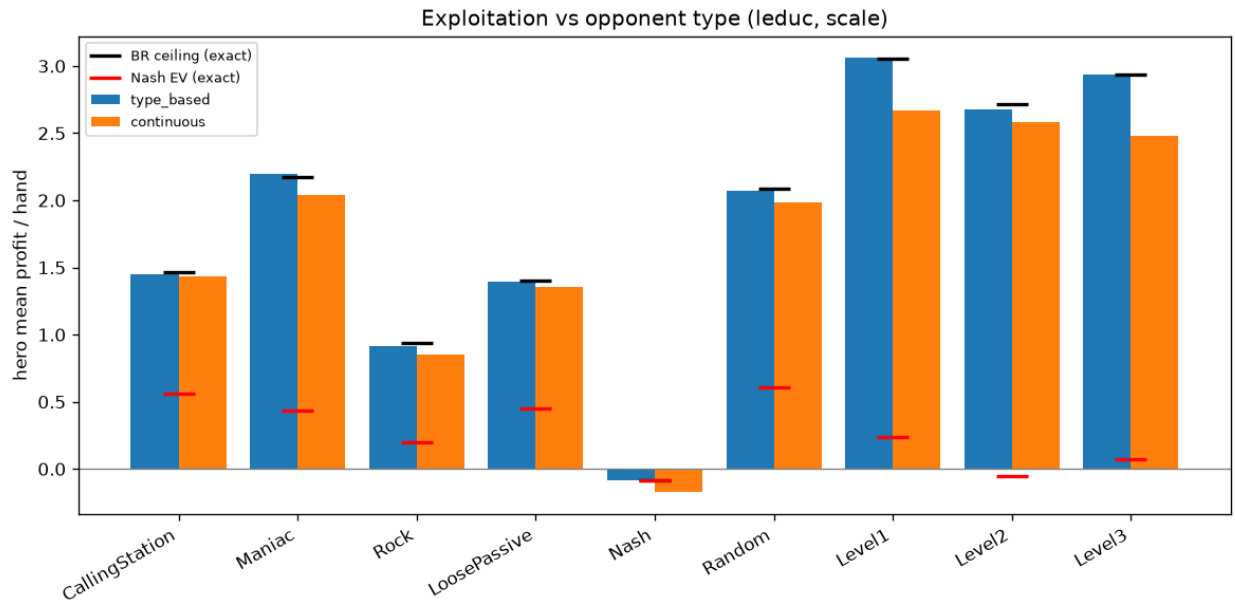


Figure 8: Exploitation vs opponent type (Leduc, scale): type-based (blue) hugs the exact BR ceiling for every type; continuous (orange) tracks close but sits below for the hardest-to-fit types; both drop below zero against Nash.

Two results carry the thesis message:

1. **You cannot exploit an equilibrium.** Against the Nash opponent every model earns $\approx$ the (negative) Nash EV, never more — the exact ceiling is essentially the game value. This is the sanity check that the exploitation is *real* and not an artifact.
2. **A confident-but-underfit model makes you exploitable.** On Leduc the continuous model *loses* to Nash (-0.175 vs the -0.083 ceiling) — with imperfect data it best-responds to a *wrong* estimate

of an unexploitable opponent, opening a leak in its own play. (At the smaller smoke sample this was a starker -0.40 ; more data shrinks but does not remove it.) This is the exploitation-vs-safety tension made concrete, and the direct motivation for Step 08’s safe (KL-regularized) exploitation.

12. Non-stationarity — the two-sided finding

The opponent switches style mid-match; we compare a **static** continuous model against one with **change-point forgetting** (detect a style change $\rightarrow$ reset the model $\rightarrow$ drop to safe play $\rightarrow$ re-learn). The result is **scenario-dependent**, which is itself the finding:

Game	Switch (at hand 10k of 20k)	static (after switch)	change-point (after switch)
Kuhn	TightPassive $\rightarrow$ LooseAggressive	-0.116	$+0.226$
Leduc	Rock $\rightarrow$ Maniac	$+1.940$	$+0.525$

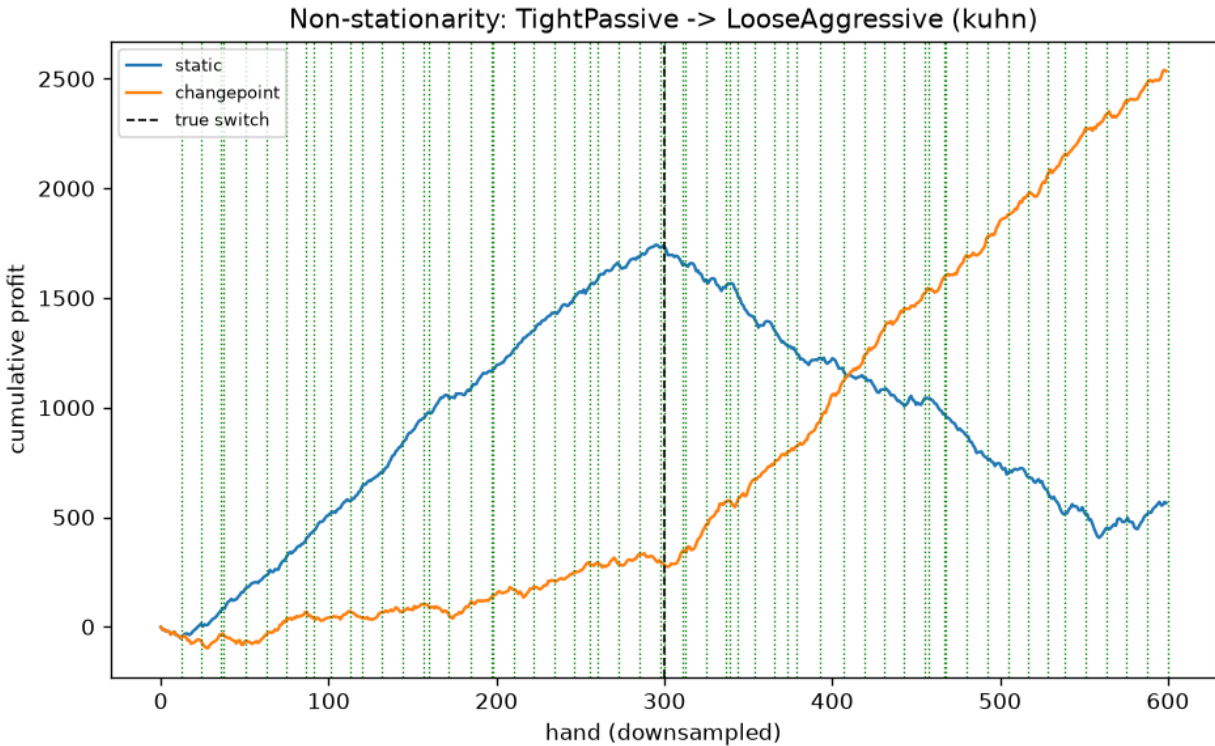


Figure 9: Non-stationarity (Kuhn): after the switch the static model bleeds (exploiting a phantom), while change-point detection resets and recovers.

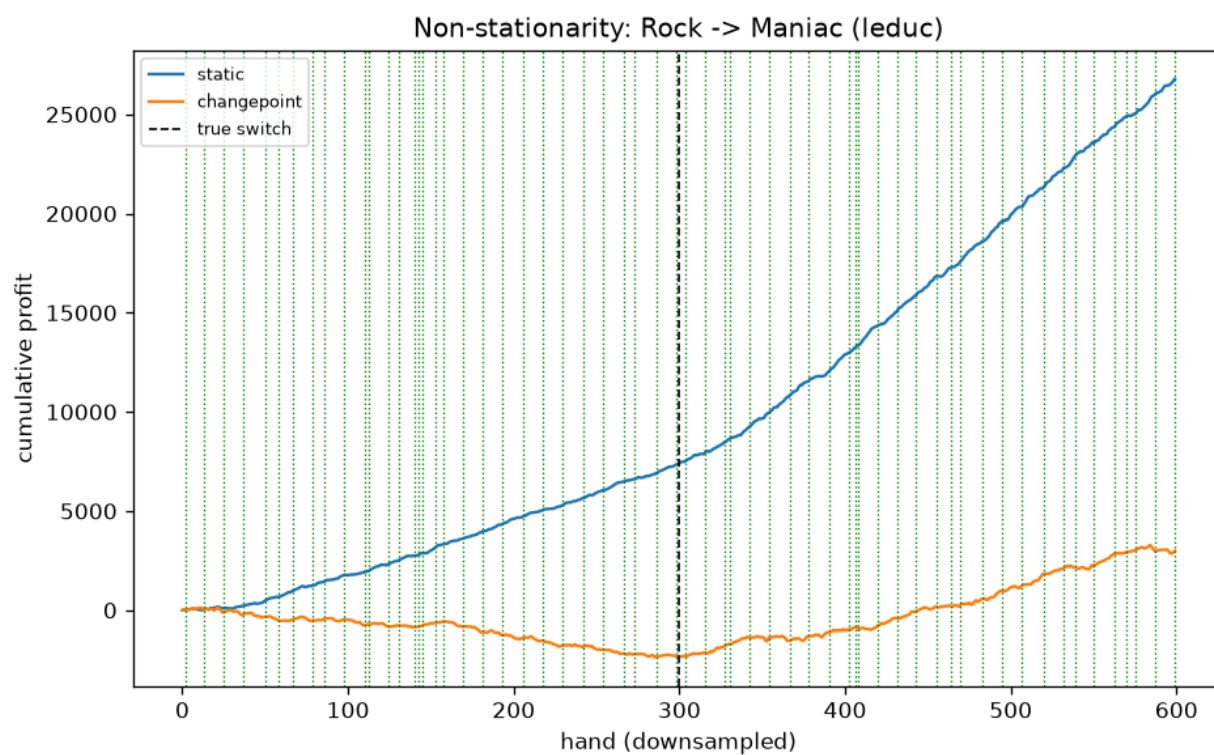


Figure 10: Non-stationarity (Leduc): here the static model wins — the Maniac is so exploitable that continuous adaptation captures it, and the detector’s false-positive resets cost more than they save.

- **Kuhn — change-point wins.** The stale anti-rock strategy (bluff-heavy) *actively loses* to the new maniac (who calls everything), so the static model goes negative; detecting the switch and re-learning recovers to +0.20.
- **Leduc — change-point loses.** The maniac leaks 2+/hand, so the continuously-adapting static model exploits it well without any reset; meanwhile the detector fires several **false positives** during the stable phase (visible as the reset dips in the plot), each dropping to safe play and discarding data.

The lesson: change-point forgetting rescues you exactly when a stale model is *harmful*, but its naive form — a low-signal Bernoulli detector that occasionally false-triggers, plus a full reset-to-safe on every detection — can underperform simple continuous adaptation when the new opponent is exploitable enough that staleness costs little. Cheaper reactions (partial forgetting instead of a hard reset; a lower false-positive rate) are the obvious next step, and a clean motivation for the thesis’s non-stationarity thread.

13. Were the samples large enough? — statistical adequacy

The honest answer is **yes for detection and exploitation, no for non-stationarity**. Because every experiment logs its per-seed values, we can quantify this rather than assert it. (Sizing: detection = 5 seeds $\times$ 4 000 hands; exploitation = 5 seeds $\times$ 20 000 hands; non-stationarity = 1 seed $\times$ 20 000 hands.)

Exploitation — comfortably adequate. The standard error of the mean profit/hand across the 5 seeds is tiny relative to the effects we claim:

Game	typical SE (per-hand)	type-based gap to ceiling	continuous shortfall vs type-based
Kuhn	0.000–0.005	within $\approx$ 1 SE everywhere	$\leq$ 1 SE (they coincide)
Leduc	0.007–0.040	within $\approx$ 1 SE everywhere	5–20 SE (e.g. Level1 3.061 vs 2.672, $\Delta = 0.389$ at SE ≈ 0.02 –0.04)

Two consequences: (1) the type-based model is **statistically indistinguishable from the exact best-response ceiling** on every type in both games — the “modeling reaches the ceiling” claim is not seed luck; and (2) the continuous model’s Leduc shortfall (§11) is a **real, $\sim$ 5–20 σ effect**, not noise — so the undersampling-of-info-sets story is a measured finding, and the Nash self-leak (–0.175 vs the –0.083 ceiling, a 0.092 gap at SE $\approx 0.015 \approx 6 \sigma$) is likewise real. In other words the sample is *more* than large enough to resolve every difference the report leans on.

Detection — adequate for the claims made. Type-based posteriors are saturated (1.000 / MAP correct) and the continuous/consistent TV distances are stable and ordered as predicted (Kuhn TV 0.004–0.030; Leduc continuous 0.11–0.36). We report point TVs rather than error bars here; that is enough to support the qualitative ordering (type-based > consistent ≥ continuous on recovery) but a fuller version would repeat detection across seeds and publish TV confidence bands — cheap to add.

Non-stationarity — under-powered (the real gap). This experiment runs a **single fixed RNG seed** (`random.Random(4242)`), so each reported number is one realization, $n = 1$, with **no error bar at all**. The direction of the two-sided finding (§12) is robust — the mechanisms (stale model actively losing on Kuhn; false-positive resets discarding data on Leduc) are structural, and the change-point count (60–61 resets against a *single* true switch) is unambiguous. But the *magnitudes* (+0.226, +0.525, +1.940) should be read as illustrative, not estimated. Repeating this experiment over the same 5 seeds is the single most valuable cheap upgrade to the run, and is listed first in §15.

14. Bugs found and fixed (first-execution engineering record)

Two correctness defects and one tractability problem in the written-but-never-run code, plus cleanups:

1. **Change-point detection was inert.** The exploiter reacted only when `changepoint_prob(window=3) > 0.5`, but that posterior mass **peaks at ≈ 0.37** after a real switch (the detection-lag mass concentrates around run-length ≈ 6 , *outside* a width-3 window), so it **never fired** — the whole non-stationarity feature was dead. Fixed by keying on the **MAP run-length collapse** ($150+ \rightarrow \approx 6$ at detection), the standard BOCPD signal. This is what makes §12 (and `validate.py`'s change-point check) meaningful.
2. **Leduc non-stationarity crashed** (`KeyError: 'TightPassive'`) — the config hard-coded Kuhn type names for the switching opponents, absent from the Leduc zoo. Fixed with **game-specific** opponents (Kuhn: `TightPassive`→`LooseAggressive`; Leduc: `Rock`→`Maniac`).
3. **The consistent model made scale intractable.** In the exploitation loop it refits a convex program *every 200 hands over a growing buffer up to 20k hands* — measured fit time grows $\sim 0.1 \text{ s} \rightarrow 12 \text{ s}$ ($N = 200 \rightarrow 19\,600$), so a single 20k-hand match runs to tens of minutes and the full sweep to many hours. The first scale run hung on the first such match for 17+ min and was killed. **Resolution:** the consistent model is evaluated where its Ganzfried-2025 strength lies — **recovery (detection)** — and left out of the online exploitation loop (which type-based + continuous already characterise against exact ceilings). Detection runs it on Kuhn (one bounded solve per item); Leduc consistent detection is deferred (its sequence-form program is large).
4. **Cleanups & run infrastructure:** silenced benign SciPy solver warnings (singular constraint Jacobian / locally-linear objective — handled correctly, result validated); git-ignored the Nash CFR cache and run logs; and added tournament observability/robustness — a 30 s progress heartbeat, per-seed/per-hand progress lines (rate-limited so fast models don't flood the log), a resum-

able checkpoint (per-item, incl. per-seed, autosaved every 5 min so a crash resumes rather than restarts), and a timestamped tee'd log file.

Per WORKFLOW §0.1, each mismatch was traced to a real cause before being called correct-or-fixed.

15. Concerns and threats to validity

Ranked by how much they qualify the conclusions:

1. **Non-stationarity is $n = 1$ (quantified in §13).** The clearest statistical gap. The *finding* holds (structural, and the 60/61-reset count is decisive), but the magnitudes are single realizations. **Fix:** loop the experiment over the existing 5 seeds — a few minutes of compute.
 2. **The change-point detector is too trigger-happy (§12).** It fires 60–61 times against one true switch — roughly one false reset every ~ 330 hands during a *stationary* regime. On Leduc this actively costs profit (each reset discards the learned model and drops to safe play). This is a genuine quality defect in the naive Bernoulli-BOCPD + hard-reset design, not just a tuning nit; it is the reason change-point *loses* on Leduc.
 3. **Consistent-model exploitation is unmeasured by design (§14.3).** Excluding it keeps scale tractable, but it means the report's exploitation comparison is type-based vs continuous only. Its *recovery* is measured (detection), so the omission is scoped, not silent — but a reader wanting a three-way exploitation table won't find one.
 4. **Leduc consistent detection is deferred.** The sequence-form convex program is large on Leduc; we have consistent-model recovery only on Kuhn. The cross-game recovery claim therefore rests on Kuhn.
 5. **Well-specified detection is an easy case.** Every true type is also a candidate in the type-based menu, so posterior = 1.000 is *expected*; it validates the machinery, not robustness to an opponent *outside* the menu — which is exactly where the continuous/consistent models are supposed to earn their keep (and, on Leduc, where continuous visibly struggles).
 6. **No OpenSpiel cross-check.** pyspiel does not install cleanly on this Windows box; the exact internal Nash checks (value = $-1/18$, NashConv ≈ 0.002) substitute. Adequate for correctness, but an independent implementation would be stronger.
 7. **Two small games, self-play data.** Kuhn/Leduc are exact-solvable *because* they are tiny; the partial-observability and undersampling effects that dominate Leduc will only intensify in larger games. Everything here is a controlled proof-of-concept, not a scaling claim.
-

16. New research directions this opened

The value of running the code (vs. only writing it) is that several of these are now *motivated by measured behavior on this machine*, not just anticipated:

- **Safe (KL-regularized) exploitation — directly motivated.** The continuous model’s Nash self-leak (§11: it *loses* to an unexploitable opponent because it best-responds to a wrong estimate) is the exploitation-vs-safety tension made empirical. This is the concrete hand-off into **Step 08** and thesis **Contribution #2** (Multi-Agent Safe Exploitation): bound the best response’s deviation from Nash by the model’s own confidence, so an underfit model cannot open a leak.
- **Confidence-scaled / partial forgetting instead of hard reset.** The §12 two-sided result says the *reaction* to a detected change matters as much as the detection: reset-to-safe is too blunt when the new opponent is exploitable. A forgetting factor (down-weight old observations) or a reset whose depth scales with detection confidence is a clean, testable improvement.
- **A better change-point signal.** The current low-signal Bernoulli aggression detector is the false-positive source (§15.2). Richer signals — per-info-set likelihood-ratio monitoring, or a multivariate BOCPD over several behavioral features — are a self-contained sub-study, and one that maps onto real-time opponent-shift detection in the target domain.
- **Misspecified / out-of-menu opponents.** Because well-specified detection is saturated (§15.5), the interesting regime is opponents *not* in the type menu (mixtures, drifting, or adversarially-crafted). This is where the non-parametric (continuous) and sequence-form (consistent) models should separate from type-based — and where a “none of my types fit” detector (open-world type discovery) becomes a research question in its own right. Part I §4.2/§5 already shows the smallest version of this problem on Kuhn.
- **Online/incremental consistent model.** The tractability wall (§14.3) is an *engineering* limit, not a fundamental one: warm-starting the convex solve from the previous fit and caching hand-terms incrementally would plausibly bring a Ganzfried-2025 sequence-form model into the online exploitation loop — turning a currently-detection-only method into a usable real-time exploiter. That is a concrete, publishable systems contribution.
- **Undersampling-aware recovery.** The Leduc continuous shortfall is dominated by rarely-visited info sets. Priors that share statistics across related info sets (hierarchical/abstraction-based smoothing) are the natural fix and connect opponent modeling to the abstraction literature.

REPRODUCTION

17. Reproduction

Part I — exploration experiments (from repo root, with `.venv` activated; each script prints tables to stdout and saves JSON caches + PNG figures to `implementation/step07/exploration/figures/`; the

per-experiment scripts run in under a second, the robustness sweep in a few seconds):

```
python implementation/step07/exploration/kuhn_tools.py           # shared helpers self-test
python implementation/step07/exploration/behavioral_fingerprints.py # Exp 1
python implementation/step07/exploration/exploitation_opportunity.py # Exp 2 (Nash EV vs BR EV)
python implementation/step07/exploration/bayesian_type_detector.py # Exp 3 (5 hidden opponents)
python implementation/step07/exploration/mixture_recovery.py      # Exp 4 (EM, 50/50 & 70/30)
python implementation/step07/exploration/robustness_sweep.py      # §4.3 sweep (500 hands × 300 seeds)
```

Part II — implementation validation + tournament (from `implementation/step07/implementation/`, with the repo `.venv` active; needs `numpy`, `scipy`, `matplotlib`):

```
python validate.py           # 8/8 PASS
python tournament.py --config smoke # Kuhn quick pass (~3 s)
python tournament.py --config scale # full headline run: Kuhn ~2 min, Leduc ~14 min
python tournament.py --config scale --fresh # ignore any checkpoint and recompute from scratch
```

individual model/plumbing self-tests each have a `__main__` (e.g. `python changepoint.py`)

A long run prints a timestamped progress line per work-item (with seed and hand counts) plus a 30 s heartbeat, tees everything to `logs/tournament_<config>_<startdate>.log` (git-ignored), and checkpoints partial results — per item, including **per seed**, autosaved every 5 min — to `_cache/tournament_<config>.ckpt.json`, so an interrupted run **resumes where it left off** on the next invocation. Results write to `results/*.json`; plots to `plots/*.png`.

All figures reproduced in this report are copied to `deliverables/reports/step07/figures/` (exploration figures under their original names; implementation figures prefixed `impl_`).